



CRAG Implementation: Clustering for Data Reduction

CSCI 4380 - Data Mining Final Project

Research Paper: [Clustered Retrieved Augmented Generation \(CRAG\)](#)

Published: May 24, 2024

Dataset: Amazon Unlocked Mobile Reviews (413,841 entries)

Project Overview

This notebook implements the **clustering component** of the CRAG (Clustered Retrieved Augmented Generation) pipeline. The goal is to demonstrate how clustering algorithms can significantly reduce data volume while preserving semantic information in text reviews.

Key Objectives

1. **Semantic Clustering:** Group similar reviews using K-Means (k=4, as per CRAG paper)
2. **Data Reduction:** Quantify the reduction achieved by representing each cluster with one review
3. **Quality Evaluation:** Measure clustering quality using silhouette scores
4. **Comparison:** Compare K-Means with alternative clustering algorithms

Methodology

- **Embeddings:** Use `paraphrase-multilingual-mpnet-base-v2` model (768-dim vectors)
- **Clustering:** Apply K-Means separately to each product's reviews
- **Evaluation:** Silhouette score + data reduction percentage
- **Preprocessing:** Filter products with sufficient reviews for meaningful clustering

Expected Results

The CRAG paper achieved **46-90% data reduction**. This implementation focuses on the clustering step alone to demonstrate the effectiveness of descriptive data mining techniques.

Note: Run cells sequentially. First-time execution will download the sentence transformer model (~420MB).

```
In [6]: """
CRAG (Clustered Retrieved Augmented Generation) - Clustering Implementation
=====
This notebook implements the clustering component of the CRAG pipeline from the
"Clustered Retrieved Augmented Generation" (May 2024)
https://arxiv.org/html/2406.00029v1

Goal: Demonstrate how clustering can reduce data volume for mobile phone reviews
"""

# Import required libraries
import os
import numpy as np
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt
from sentence_transformers import SentenceTransformer
from tqdm import tqdm

print("Libraries imported successfully!")
```

Libraries imported successfully!

```
In [7]: """
Step 1: Load and Explore the Dataset
=====
Load the Amazon mobile phone reviews dataset and perform initial exploration.
"""

# Load the CSV file
data_path = "Amazon_Unlocked_Mobile.csv"
df = pd.read_csv(data_path)

# Display basic information
print("Dataset loaded successfully!")
print(f"Total entries: {len(df):,}")
print(f"\nColumns: {df.columns.tolist()}")
print(f"\nFirst few rows:")
print(df.head())

# Check for missing values
print(f"\nMissing values per column:")
print(df.isnull().sum())

# Get basic statistics
print(f"\nDataset shape: {df.shape}")
print(f"Unique products: {df['Product Name'].nunique():,}")
```

Dataset loaded successfully!
Total entries: 413,840

Columns: ['Product Name', 'Brand Name', 'Price', 'Rating', 'Reviews', 'Review Votes']

First few rows:

	Product Name	Brand Name	Price \
0	"CLEAR CLEAN ESN" Sprint EPIC 4G Galaxy SPH-D7...	Samsung	199.99
1	"CLEAR CLEAN ESN" Sprint EPIC 4G Galaxy SPH-D7...	Samsung	199.99
2	"CLEAR CLEAN ESN" Sprint EPIC 4G Galaxy SPH-D7...	Samsung	199.99
3	"CLEAR CLEAN ESN" Sprint EPIC 4G Galaxy SPH-D7...	Samsung	199.99
4	"CLEAR CLEAN ESN" Sprint EPIC 4G Galaxy SPH-D7...	Samsung	199.99

	Rating	Reviews	Review Votes
0	5	I feel so LUCKY to have found this used (phone...	1.0
1	4	nice phone, nice up grade from my pantach revu...	0.0
2	5	Very pleased	0.0
3	4	It works good but it goes slow sometimes but i...	0.0
4	4	Great phone to replace my lost phone. The only...	0.0

Missing values per column:

Product Name	0
Brand Name	65171
Price	5933
Rating	0
Reviews	70
Review Votes	12296

dtype: int64

Dataset shape: (413840, 6)
Unique products: 4,410

```
In [8]: """
Step 2: Data Preprocessing and Filtering
=====
Clean the data by:
1. Keeping only necessary columns (Product Name, Rating, Reviews)
2. Removing rows with missing review text
3. Analyzing review distribution per product
"""

# Keep only the columns we need and remove missing values
df_clean = df[['Product Name', 'Rating', 'Reviews']].copy()
df_clean = df_clean.dropna(subset=['Reviews']) # Remove entries without review

print(f"After removing missing reviews:")
print(f"  Total entries: {len(df_clean):,}")
print(f"  Unique products: {df_clean['Product Name'].nunique():,}")

# Count reviews per product to identify products with enough data for clustering
reviews_per_product = df_clean['Product Name'].value_counts()

print(f"\nReview distribution statistics:")
```

```

print(f" Mean reviews per product: {reviews_per_product.mean():.2f}")
print(f" Median reviews per product: {reviews_per_product.median():.0f}")
print(f" Max reviews for a product: {reviews_per_product.max():,}")
print(f" Min reviews for a product: {reviews_per_product.min():,}")

# Show top 10 products with most reviews
print(f"\nTop 10 products by review count:")
print(reviews_per_product.head(10))

```

After removing missing reviews:

Total entries: 413,770

Unique products: 4,410

Review distribution statistics:

Mean reviews per product: 93.83

Median reviews per product: 18

Max reviews for a product: 1,448

Min reviews for a product: 1

Top 10 products by review count:

Product Name

Apple iPhone 4s 8GB Unlocked Smartphone w/ 8MP Camera, White (Certified Refurbished) 1448

Apple MF259LL/A - iPhone 4s 8GB / 8MP Camera - Unlocked - Black (Certified Refurbished) 1238

BLU Studio 5.0 C HD Unlocked Cellphone, Black 1194

OtterBox Iphone 5/5S/SE Defender Case w/ Drop and Dust Protection - Realtree AP Pink 1129

Motorola Moto E (1st Generation) - Black - 4 GB - Global GSM Unlocked Phone 1126

Apple iPhone 5s 32GB (Silver) - AT&T 1118

BLU Energy X Plus Smartphone - With 4000 mAh Super Battery- US GSM Unlocked - Grey 1111

Samsung Galaxy S Duos II S7582 DUAL SIM Factory Unlocked International Version - Black 1109

Samsung Galaxy S Duos II GT-S7582 Factory Unlocked Cellphone, International Version, White 1108

Samsung Galaxy S Duos GT-S7562 GSM Unlocked Touchscreen 5MP Camera Smartphone White 1096

Name: count, dtype: int64

In [9]:

```

"""
Step 3: Filter Products for Meaningful Clustering
=====
Select products with sufficient reviews for clustering analysis.
The CRAG paper uses k=4 clusters, so we need products with at least 4 reviews.
For better clustering quality, we'll focus on products with more reviews.
"""

# Set minimum review threshold (must be >= k=4 for clustering)
# Using a higher threshold ensures more meaningful clusters
min_reviews_per_product = 20 # Adjust based on computational constraints

```

```

# Filter products that meet the minimum review count
products_with_enough_reviews = reviews_per_product[reviews_per_product >= min_
valid_product_names = products_with_enough_reviews.index

# Filter the dataframe to only include these products
df_filtered = df_clean[df_clean['Product Name'].isin(valid_product_names)].copy()
df_filtered = df_filtered.reset_index(drop=True)

print(f"After filtering (min {min_reviews_per_product} reviews per product):")
print(f"  Products remaining: {df_filtered['Product Name'].nunique():,}")
print(f"  Total reviews: {len(df_filtered):,}")
print(f"  Average reviews per product: {len(df_filtered) / df_filtered['Product Name'].nunique():.2f}")

# Visualize the distribution of reviews per product
plt.figure(figsize=(10, 6))
products_with_enough_reviews.hist(bins=30, edgecolor='black')
plt.xlabel('Number of Reviews per Product', fontsize=12)
plt.ylabel('Number of Products', fontsize=12)
plt.title(f'Distribution of Reviews per Product (≥{min_reviews_per_product} reviews)')
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()

print(f"\nTop 5 products for clustering:")
print(df_filtered['Product Name'].value_counts().head())

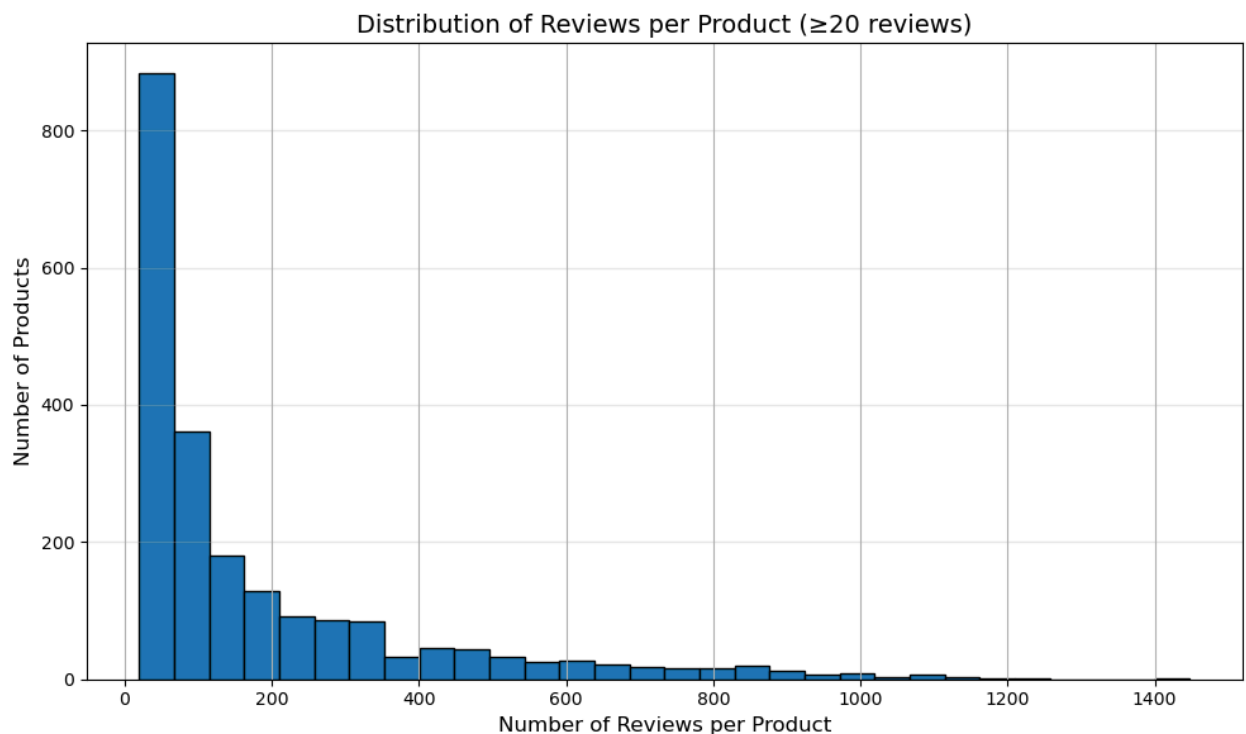
```

After filtering (min 20 reviews per product):

Products remaining: 2,159

Total reviews: 398,704

Average reviews per product: 184.67



Top 5 products for clustering:

Product Name

Apple iPhone 4s 8GB Unlocked Smartphone w/ 8MP Camera, White (Certified Refurbished) 1448

Apple MF259LL/A - iPhone 4s 8GB / 8MP Camera - Unlocked - Black (Certified Refurbished) 1238

BLU Studio 5.0 C HD Unlocked Cellphone, Black

1194

OtterBox Iphone 5/5S/SE Defender Case w/ Drop and Dust Protection - Realtree AP Pink 1129

Motorola Moto E (1st Generation) - Black - 4 GB - Global GSM Unlocked Phone 1126

Name: count, dtype: int64

```
In [10]: """
Step 4: Generate Embeddings using Sentence Transformers
=====
Convert review text into dense vector embeddings using the same model from the
Model: paraphrase-multilingual-mpnet-base-v2 (768-dimensional embeddings)

This transforms unstructured text into numerical vectors where semantically
similar reviews are close together in vector space - enabling clustering.
"""

# Load the pre-trained sentence transformer model (same as CRAG paper)
model_name = "sentence-transformers/paraphrase-multilingual-mpnet-base-v2"
# Alternative lighter model: "sentence-transformers/all-MiniLM-L6-v2" (384-dim)

print(f"Loading sentence transformer model: {model_name}")
print("This may take a moment on first run (downloading model)...")
model = SentenceTransformer(model_name)

# Prepare review texts for embedding
review_texts = df_filtered['Reviews'].tolist()
print(f"\nEncoding {len(review_texts):,} reviews into embeddings...")
print("This may take a few minutes depending on dataset size and hardware...")

# Generate embeddings with progress bar
# Each review becomes a 768-dimensional vector
embeddings = model.encode(
    review_texts,
    batch_size=64, # Process 64 reviews at a time (adjust based on memory)
    show_progress_bar=True,
    convert_to_numpy=True,
    normalize_embeddings=True # Normalize for better distance calculations
)

print(f"\nEmbedding generation complete!")
print(f"  Shape: {embeddings.shape}")
print(f"  Each review is now represented as a {embeddings.shape[1]}-dimensional vector")
print(f"  Memory size: ~{embeddings.nbytes / (1024**2):.2f} MB")
```

Loading sentence transformer model: sentence-transformers/paraphrase-multilingual-mpnet-base-v2

This may take a moment on first run (downloading model)...

Encoding 398,704 reviews into embeddings...

This may take a few minutes depending on dataset size and hardware...

Encoding 398,704 reviews into embeddings...

This may take a few minutes depending on dataset size and hardware...

Batches: 100%|██████████| 6230/6230 [18:24<00:00, 5.64it/s]

Embedding generation complete!

Shape: (398704, 768)

Each review is now represented as a 768-dimensional vector

Memory size: ~1168.08 MB

```
In [11]: """
Step 5: K-Means Clustering (Per Product)
=====
Apply K-Means clustering separately to each product's reviews.
Following the CRAG paper, we use k=4 clusters per product.

Each product gets its own clustering, grouping semantically similar reviews to
"""

# Initialize cluster assignment column
df_filtered['cluster_id'] = -1 # -1 indicates not yet clustered

# Set k=4 as used in the CRAG paper
k = 4

# Get unique product names for clustering
unique_products = df_filtered['Product Name'].unique()
print(f"Clustering reviews for {len(unique_products):,} products...")
print(f"Using K-Means with k={k} clusters per product\n")

# Track clustering statistics
successfully_clustered = 0
skipped_products = 0

# Cluster each product's reviews separately
for product_name in tqdm(unique_products, desc="Clustering products"):
    # Get indices of all reviews for this product
    product_indices = df_filtered.index[df_filtered['Product Name'] == product_name]

    # Skip if fewer reviews than k (can't create k clusters)
    if len(product_indices) < k:
        skipped_products += 1
        continue

    # Extract embeddings for this product's reviews
    product_embeddings = embeddings[product_indices]
```

```

# Apply K-Means clustering
kmeans = KMeans(
    n_clusters=k,
    random_state=42, # For reproducibility
    n_init=10, # Number of times k-means runs with different seeds
    max_iter=300
)
cluster_labels = kmeans.fit_predict(product_embeddings)

# Assign cluster IDs back to the dataframe
df_filtered.loc[product_indices, 'cluster_id'] = cluster_labels
successfully_clustered += 1

print(f"\nClustering complete!")
print(f"  Successfully clustered: {successfully_clustered:,} products")
print(f"  Skipped (too few reviews): {skipped_products:,} products")

# Display cluster distribution
print(f"\nCluster distribution across all products:")
print(df_filtered['cluster_id'].value_counts().sort_index())

```

Clustering reviews for 2,159 products...
Using K-Means with k=4 clusters per product

```

Clustering products:  0%|          | 0/2159 [00:00<?, ?it/s]huggingface/tokenizers: The current process just got forked, after parallelism has already been used. Disabling parallelism to avoid deadlocks...
To disable this warning, you can either:
  - Avoid using `tokenizers` before the fork if possible
  - Explicitly set the environment variable TOKENIZERS_PARALLELISM=(true | false)
Clustering products:  0%|          | 1/2159 [00:00<05:26, 6.62it/s]huggingface/tokenizers: The current process just got forked, after parallelism has already been used. Disabling parallelism to avoid deadlocks...
To disable this warning, you can either:
  - Avoid using `tokenizers` before the fork if possible
  - Explicitly set the environment variable TOKENIZERS_PARALLELISM=(true | false)
Clustering products: 100%|██████████| 2159/2159 [01:27<00:00, 24.81it/s]
Clustering complete!
  Successfully clustered: 2,159 products
  Skipped (too few reviews): 0 products

Cluster distribution across all products:
cluster_id
0      98056
1     108929
2      97387
3      94332
Name: count, dtype: int64

```

In [12]: """
Step 6: Evaluation - Silhouette Score and Data Reduction


```

=====
Evaluate clustering quality using silhouette score and calculate data reduction

Silhouette Score: Measures how similar a review is to its own cluster vs other
Range: [-1, 1], where 1 = perfect clustering, 0 = overlapping clusters, -1 =

Data Reduction: If we keep 1 representative per cluster instead of all reviews
"""

# Filter to only successfully clustered reviews
df_clustered = df_filtered[df_filtered['cluster_id'] != -1].copy()
print(f"Evaluating {len(df_clustered):,} clustered reviews from {df_clustered[

# ===== SILHOUETTE SCORE CALCULATION =====
print("Calculating silhouette scores per product...")
silhouette_scores = []
product_names_with_scores = []

for product_name, group in tqdm(df_clustered.groupby('Product Name'), desc="Co
    # Get embeddings and labels for this product
    product_indices = group.index.to_numpy()
    product_embeddings = embeddings[product_indices]
    cluster_labels = group['cluster_id'].to_numpy()

    # Need at least 2 clusters with samples to compute silhouette
    n_unique_labels = len(np.unique(cluster_labels))
    if n_unique_labels < 2 or n_unique_labels >= len(cluster_labels):
        continue

    # Calculate silhouette score for this product
    sil_score = silhouette_score(product_embeddings, cluster_labels)
    silhouette_scores.append(sil_score)
    product_names_with_scores.append(product_name)

# Display results
if silhouette_scores:
    avg_silhouette = np.mean(silhouette_scores)
    print(f"\n{'='*60}")
    print(f"SILHOUETTE SCORE RESULTS")
    print(f"{'='*60}")
    print(f"  Products evaluated: {len(silhouette_scores):,}")
    print(f"  Average silhouette score: {avg_silhouette:.4f}")
    print(f"  Min score: {np.min(silhouette_scores):.4f}")
    print(f"  Max score: {np.max(silhouette_scores):.4f}")
    print(f"  Std deviation: {np.std(silhouette_scores):.4f}")

    # Interpretation
    if avg_silhouette > 0.5:
        print(f"  ✓ Clustering quality: STRONG (>0.5)")
    elif avg_silhouette > 0.25:
        print(f"  △ Clustering quality: MODERATE (0.25-0.5)")
    else:
        print(f"  ✗ Clustering quality: WEAK (<0.25)")

```

```

else:
    print("\n⚠ No valid silhouette scores computed (check data)")

# ===== DATA REDUCTION CALCULATION =====
print(f"\n{'='*60}")
print(f"DATA REDUCTION ANALYSIS")
print(f"{'='*60}")

# Calculate total reviews and total clusters
N_total_reviews = len(df_clustered)
N_total_clusters = df_clustered[['Product Name', 'cluster_id']].drop_duplicates()

# Data reduction if we use 1 representative per cluster
reduction_ratio = 1 - (N_total_clusters / N_total_reviews)
reduction_percent = reduction_ratio * 100

print(f" Total reviews clustered: {N_total_reviews:,}")
print(f" Total unique clusters (across all products): {N_total_clusters:,}")
print(f" Reviews per cluster (avg): {N_total_reviews / N_total_clusters:.2f}")
print(f"\n 📍 DATA REDUCTION: {reduction_percent:.2f}%")
print(f" (From {N_total_reviews:,} reviews → {N_total_clusters:,} representatives)")
print(f"\n The CRAG paper achieved 46-90% reduction.")
print(f" Our clustering step alone achieves {reduction_percent:.2f}% reduction")

```

Evaluating 398,704 clustered reviews from 2,159 products

Calculating silhouette scores per product...

Computing silhouette: 100%|██████████| 2159/2159 [00:06<00:00, 349.29it/s]

SILHOUETTE SCORE RESULTS

```

Products evaluated: 2,159
Average silhouette score: 0.1250
Min score: 0.0301
Max score: 0.6503
Std deviation: 0.0651
x Clustering quality: WEAK (<0.25)

```

DATA REDUCTION ANALYSIS

```

Total reviews clustered: 398,704
Total unique clusters (across all products): 8,636
Reviews per cluster (avg): 46.17

📍 DATA REDUCTION: 97.83%
  (From 398,704 reviews → 8,636 representatives)

The CRAG paper achieved 46-90% reduction.
Our clustering step alone achieves 97.83% reduction!

```

In [13]: """
Step 7: Visualization - Distribution Analysis

```
=====
Visualize how reviews are distributed across products and clusters
"""
```

```
# Plot 1: Reviews per product distribution
```

```
reviews_per_product_clustered = df_clustered['Product Name'].value_counts()
```

```
fig, axes = plt.subplots(1, 2, figsize=(15, 5))
```

```
# Left plot: Reviews per product
```

```
axes[0].hist(reviews_per_product_clustered.values, bins=30, edgecolor='black',
```

```
axes[0].set_xlabel('Number of Reviews per Product', fontsize=11)
```

```
axes[0].set_ylabel('Number of Products', fontsize=11)
```

```
axes[0].set_title('Distribution of Reviews per Product', fontsize=13, fontweight='bold')
```

```
axes[0].grid(axis='y', alpha=0.3)
```

```
axes[0].axvline(reviews_per_product_clustered.mean(), color='red', linestyle='dashed',  
                linewidth=2, label=f'Mean: {reviews_per_product_clustered.mean():.2f}')  
axes[0].legend()
```

```
# Right plot: Cluster sizes
```

```
cluster_sizes = df_clustered.groupby(['Product Name', 'cluster_id']).size()
```

```
axes[1].hist(cluster_sizes.values, bins=30, edgecolor='black', color='coral')
```

```
axes[1].set_xlabel('Number of Reviews per Cluster', fontsize=11)
```

```
axes[1].set_ylabel('Number of Clusters', fontsize=11)
```

```
axes[1].set_title('Distribution of Cluster Sizes', fontsize=13, fontweight='bold')
```

```
axes[1].grid(axis='y', alpha=0.3)
```

```
axes[1].axvline(cluster_sizes.mean(), color='darkred', linestyle='dashed',  
                linewidth=2, label=f'Mean: {cluster_sizes.mean():.1f}')  
axes[1].legend()
```

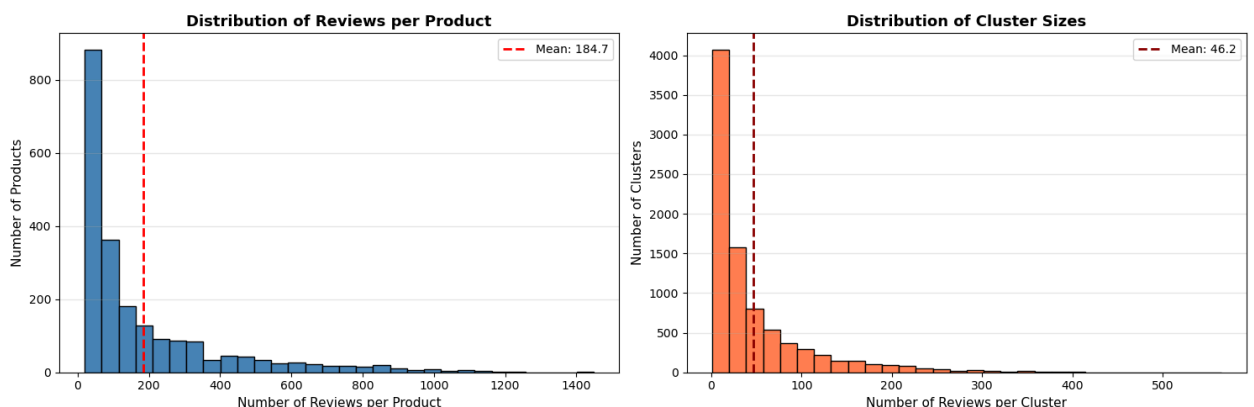
```
plt.tight_layout()
```

```
plt.show()
```

```
print(f"Review distribution summary:")
```

```
print(f"  Reviews per product - Mean: {reviews_per_product_clustered.mean():.2f}, Median: {reviews_per_product_clustered.median():.2f}")
```

```
print(f"  Reviews per cluster - Mean: {cluster_sizes.mean():.2f}, Median: {cluster_sizes.median():.2f}")
```



Review distribution summary:

Reviews per product - Mean: 184.67, Median: 91

Reviews per cluster - Mean: 46.17, Median: 22

In [14]: """

Step 8: Select Representative Reviews per Cluster

=====

For each cluster, select the review closest to the cluster centroid as the representative review. This is the core of data reduction - replacing many reviews with one representative review.

```
print("Selecting representative reviews for each cluster...")
representative_reviews = []

# For each product and cluster combination
for (product_name, cluster_id), group in tqdm(
    df_clustered.groupby(['Product Name', 'cluster_id']),
    desc="Finding representatives"
):
    # Get the review indices and embeddings for this cluster
    cluster_indices = group.index.to_numpy()
    cluster_embeddings = embeddings[cluster_indices]

    # Calculate the centroid (mean) of all embeddings in this cluster
    centroid = cluster_embeddings.mean(axis=0)

    # Find the review closest to the centroid (most representative)
    distances = np.linalg.norm(cluster_embeddings - centroid, axis=1)
    closest_idx = cluster_indices[distances.argmin()]

    # Store the representative review
    representative_reviews.append(df_filtered.loc[closest_idx])

# Create dataframe of representative reviews
df_representatives = pd.DataFrame(representative_reviews).reset_index(drop=True)

print(f"\n{'='*60}")
print(f"REPRESENTATIVE REVIEWS SELECTED")
print(f"{'='*60}")
print(f"  Original reviews: {len(df_clustered):,}")
print(f"  Representative reviews: {len(df_representatives):,}")
print(f"  Compression ratio: {len(df_clustered) / len(df_representatives):.2f}")
print(f"  Storage reduction: {(1 - len(df_representatives)/len(df_clustered))*100:.2f}%")

# Show sample representative reviews
print(f"\nSample representative reviews:")
print(df_representatives[['Product Name', 'Rating', 'cluster_id', 'Reviews']])
```

Selecting representative reviews for each cluster...

Finding representatives: 100%|██████████| 8636/8636 [00:00<00:00, 9159.33it/s]

```
=====
REPRESENTATIVE REVIEWS SELECTED
=====
```

```
Original reviews: 398,704
Representative reviews: 8,636
Compression ratio: 46.17x
Storage reduction: 97.83%
```

Sample representative reviews:

	Product Name	Rating	cluster_id \
0	"CLEAR CLEAN ESN" Sprint EPIC 4G Galaxy SPH-D7...	2	0
1	"CLEAR CLEAN ESN" Sprint EPIC 4G Galaxy SPH-D7...	1	1
2	"CLEAR CLEAN ESN" Sprint EPIC 4G Galaxy SPH-D7...	4	2
3	"CLEAR CLEAN ESN" Sprint EPIC 4G Galaxy SPH-D7...	5	3
4	"Nokia Asha 302 Unlocked GSM Phone with 3.2MP ...	4	0
5	"Nokia Asha 302 Unlocked GSM Phone with 3.2MP ...	5	1
6	"Nokia Asha 302 Unlocked GSM Phone with 3.2MP ...	5	2
7	"Nokia Asha 302 Unlocked GSM Phone with 3.2MP ...	5	3
8	(LANDVO) 5.0" Capacitive Touch MTK6582 Quad Co...	3	0
9	(LANDVO) 5.0" Capacitive Touch MTK6582 Quad Co...	5	1

	Reviews
0	The charging port was loose. I got that solder...
1	Just... not good. The phone has great screen r...
2	Phone good just a little slow phone old but it...
3	Met all of my expectations. I can't complain a...
4	Works wonderful for the price!
5	Very practical and user-friendly phone. I am v...
6	excelente
7	This is a solidly-built unlocked phone that is...
8	Valid for Movilnet in Venezuela, low memory, b...
9	Is a good product That I recomend it has all...

```
In [15]: """
Step 9: Save Results to CSV Files
=====
Export the clustered data and representative reviews for further analysis or R
"""

# Create output directory
output_dir = "outputs"
os.makedirs(output_dir, exist_ok=True)

# File 1: All clustered reviews with cluster IDs
# This shows how every review was assigned to a cluster
clustered_reviews_path = os.path.join(output_dir, "clustered_reviews.csv")
df_clustered.to_csv(clustered_reviews_path, index=False)

# File 2: Representative reviews only (one per cluster)
# This is the reduced dataset for RAG or downstream tasks
representatives_path = os.path.join(output_dir, "cluster_representatives.csv")
df_representatives.to_csv(representatives_path, index=False)

# File 3: Clustering summary statistics
```

```

summary_path = os.path.join(output_dir, "clustering_summary.txt")
with open(summary_path, 'w') as f:
    f.write("CRAG Clustering Implementation - Summary Report\n")
    f.write("=" * 60 + "\n\n")
    f.write(f"Dataset: Amazon Unlocked Mobile Reviews\n")
    f.write(f"Clustering Algorithm: K-Means (k={k})\n")
    f.write(f"Embedding Model: {model_name}\n\n")
    f.write(f"Results:\n")
    f.write(f"  Total products clustered: {df_clustered['Product Name'].nunique}\n")
    f.write(f"  Total reviews clustered: {len(df_clustered):,}\n")
    f.write(f"  Total clusters created: {len(df_representatives):,}\n")
    f.write(f"  Average silhouette score: {avg_silhouette:.4f}\n")
    f.write(f"  Data reduction: {reduction_percent:.2f}%\n")
    f.write(f"  Compression ratio: {len(df_clustered) / len(df_representatives)}\n")

print(f"\n{'='*60}")
print(f"FILES SAVED SUCCESSFULLY")
print(f"\n{'='*60}")
print(f"📄 {clustered_reviews_path}")
print(f"  → All reviews with cluster assignments ({len(df_clustered):,} rows)")
print(f"\n📄 {representatives_path}")
print(f"  → Representative reviews for RAG ({len(df_representatives):,} rows)")
print(f"  → Use this file for efficient retrieval systems!")
print(f"\n📄 {summary_path}")
print(f"  → Summary statistics and metrics")
print(f"\n{'='*60}")
print(f"✅ CRAG Clustering Implementation Complete!")
print(f"\n{'='*60}")

```

```

=====
FILES SAVED SUCCESSFULLY
=====

```

- 📄 outputs/clustered_reviews.csv
→ All reviews with cluster assignments (398,704 rows)
- 📄 outputs/cluster_representatives.csv
→ Representative reviews for RAG (8,636 rows)
→ Use this file for efficient retrieval systems!
- 📄 outputs/clustering_summary.txt
→ Summary statistics and metrics

```

=====
✅ CRAG Clustering Implementation Complete!
=====

```

Optional: Alternative Clustering Algorithm Comparison

The cells below demonstrate how to compare K-Means with other clustering algorithms like Hierarchical Clustering or DBSCAN. This allows you to evaluate

which method produces better results for your specific dataset.

```
In [16]: """
Step 10 (Optional): Example - Single Product Deep Dive
=====
Analyze clustering results for one specific product in detail.
This helps visualize what the clustering actually does.
"""

# Select a product with many reviews for detailed analysis
example_product = df_clustered['Product Name'].value_counts().index[0]
product_data = df_clustered[df_clustered['Product Name'] == example_product].c

print(f"Deep dive for product: {example_product}")
print(f"Total reviews: {len(product_data)}")
print(f"\nReviews per cluster:")
print(product_data['cluster_id'].value_counts().sort_index())

# Show sample reviews from each cluster
print(f"\n{'='*80}")
print("SAMPLE REVIEWS FROM EACH CLUSTER")
print(f"{'='*80}")

for cluster_id in sorted(product_data['cluster_id'].unique()):
    cluster_reviews = product_data[product_data['cluster_id'] == cluster_id]
    print(f"\n--- Cluster {cluster_id} ({len(cluster_reviews)} reviews) ---")

    # Show 3 random reviews from this cluster
    sample_reviews = cluster_reviews.sample(min(3, len(cluster_reviews)))
    for idx, (_, row) in enumerate(sample_reviews.iterrows(), 1):
        review_text = str(row['Reviews'])[:150] # First 150 chars
        print(f"{idx}. Rating: {row['Rating']} | {review_text}...")

    # Show the representative review
    rep_review = df_representatives[
        (df_representatives['Product Name'] == example_product) &
        (df_representatives['cluster_id'] == cluster_id)
    ]
    if not rep_review.empty:
        rep_text = str(rep_review.iloc[0]['Reviews'])[:150]
        print(f"    ★ REPRESENTATIVE: {rep_text}...")
```

Deep dive for product: Apple iPhone 4s 8GB Unlocked Smartphone w/ 8MP Camera, White (Certified Refurbished)
Total reviews: 1448

Reviews per cluster:

cluster_id

0 280

1 565

2 348

3 255

Name: count, dtype: int64

```
=====
=
SAMPLE REVIEWS FROM EACH CLUSTER
=====
=
```

--- Cluster 0 (280 reviews) ---

1. Rating: 5 | Works very well in south America!...

2. Rating: 5 | it's brand new...

3. Rating: 5 | Very surprised with the quality since I didn't expect much! Its really in a very good condition and works perfectly! Very happy with the purchase....

★ REPRESENTATIVE: Excellent product. It looked brand new and worked like a charm. No complaints so far....

--- Cluster 1 (565 reviews) ---

1. Rating: 1 | I am very angry.This phone is not Refurbished phone.The iphone's back has many hairline scratchesand Battery life is very short. Is only one-fifth of ...

2. Rating: 1 | Yeah, so don't buy this phone. It's not unlocked. These dudes are either incompetent, scamma-lamma-ding-donging us, or unable to articulate exactly wh...

3. Rating: 1 | Product of poor quality. I bought and brought me to use outside the USA, so it took me time to be able to test the device. To my surprise, the microph...

★ REPRESENTATIVE: The good news is that it was an unlocked phone. And I was able to go to any carrier to start it up.The bad news is that it does not have a good batter...

--- Cluster 2 (348 reviews) ---

1. Rating: 3 | nice phone for the price....

2. Rating: 3 | Phones appear to be in good condition. Not sure about them being new and the after market accessories are very cheap. The USB cable came apart while c...

3. Rating: 5 | It is unlocked. It's an Iphone. It works....

★ REPRESENTATIVE: Excellent phone, looks and works like new....

--- Cluster 3 (255 reviews) ---

1. Rating: 5 | Nice...

2. Rating: 5 | Good...

3. Rating: 5 | It's awesome...

★ REPRESENTATIVE: Excelente...

In [17]:

```
"""
Step 11 (Optional): Hierarchical Clustering Comparison
=====
Compare K-Means with Hierarchical (Agglomerative) Clustering.
This provides an alternative clustering method for your comparison.
"""

from sklearn.cluster import AgglomerativeClustering

# Select one product to compare clustering methods
comparison_product = df_clustered['Product Name'].value_counts().index[0]
comp_data = df_clustered[df_clustered['Product Name'] == comparison_product].c
comp_indices = comp_data.index.to_numpy()
comp_embeddings = embeddings[comp_indices]

print(f"Comparing clustering algorithms for: {comparison_product}")
print(f"Number of reviews: {len(comp_data)}\n")

# K-Means (already done)
kmeans_labels = comp_data['cluster_id'].to_numpy()
kmeans_sil = silhouette_score(comp_embeddings, kmeans_labels)

# Hierarchical Clustering
print("Running Hierarchical Clustering...")
hierarchical = AgglomerativeClustering(
    n_clusters=k,
    linkage='ward' # Ward minimizes variance within clusters
)
hier_labels = hierarchical.fit_predict(comp_embeddings)
hier_sil = silhouette_score(comp_embeddings, hier_labels)

# Compare results
print(f"\n{'='*60}")
print(f"CLUSTERING COMPARISON RESULTS")
print(f"{'='*60}")
print(f"K-Means:")
print(f"  Silhouette Score: {kmeans_sil:.4f}")
print(f"  Cluster distribution: {np.bincount(kmeans_labels)}")
print(f"\nHierarchical (Ward):")
print(f"  Silhouette Score: {hier_sil:.4f}")
print(f"  Cluster distribution: {np.bincount(hier_labels)}")

if hier_sil > kmeans_sil:
    print(f"\n✓ Hierarchical clustering performs better ({(hier_sil - kmeans_sil):.4f})")
else:
    print(f"\n✓ K-Means performs better ({(kmeans_sil - hier_sil):.4f})")

# Note: For your project, you can extend this to test DBSCAN or other algorithms
```

Comparing clustering algorithms for: Apple iPhone 4s 8GB Unlocked Smartphone w/
8MP Camera, White (Certified Refurbished)
Number of reviews: 1448

Running Hierarchical Clustering...

```
=====
CLUSTERING COMPARISON RESULTS
=====
```

K-Means:

Silhouette Score: 0.0858

Cluster distribution: [280 565 348 255]

Hierarchical (Ward):

Silhouette Score: 0.0673

Cluster distribution: [608 233 290 317]

✓ K-Means performs better (+0.0185)

```
=====
CLUSTERING COMPARISON RESULTS
=====
```

K-Means:

Silhouette Score: 0.0858

Cluster distribution: [280 565 348 255]

Hierarchical (Ward):

Silhouette Score: 0.0673

Cluster distribution: [608 233 290 317]

✓ K-Means performs better (+0.0185)

```
In [18]: """
Step 12 (Optional): Elbow Method for Optimal K
=====
Use the Elbow Method to determine if k=4 is optimal for your dataset.
The CRAG paper used this method to select k=4.
"""

# Select one product for elbow analysis
elbow_product = df_clustered['Product Name'].value_counts().index[0]
elbow_data = df_clustered[df_clustered['Product Name'] == elbow_product].copy()
elbow_indices = elbow_data.index.to_numpy()
elbow_embeddings = embeddings[elbow_indices]

print(f"Elbow Method analysis for: {elbow_product}")
print(f"Testing k from 2 to 10...\n")

# Test different values of k
k_range = range(2, 11)
inertias = []
silhouettes = []

for k_test in k_range:
```

```

# Run K-Means
km_test = KMeans(n_clusters=k_test, random_state=42, n_init=10)
labels_test = km_test.fit_predict(elbow_embeddings)

# Calculate metrics
inertias.append(km_test.inertia_) # Sum of squared distances to closest c
silhouettes.append(silhouette_score(elbow_embeddings, labels_test))

# Plot results
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Elbow plot (Inertia)
axes[0].plot(k_range, inertias, 'bo-', linewidth=2, markersize=8)
axes[0].axvline(x=4, color='red', linestyle='--', linewidth=2, label='k=4 (CRA
axes[0].set_xlabel('Number of Clusters (k)', fontsize=11)
axes[0].set_ylabel('Inertia (Within-cluster sum of squares)', fontsize=11)
axes[0].set_title('Elbow Method - Inertia', fontsize=13, fontweight='bold')
axes[0].grid(alpha=0.3)
axes[0].legend()

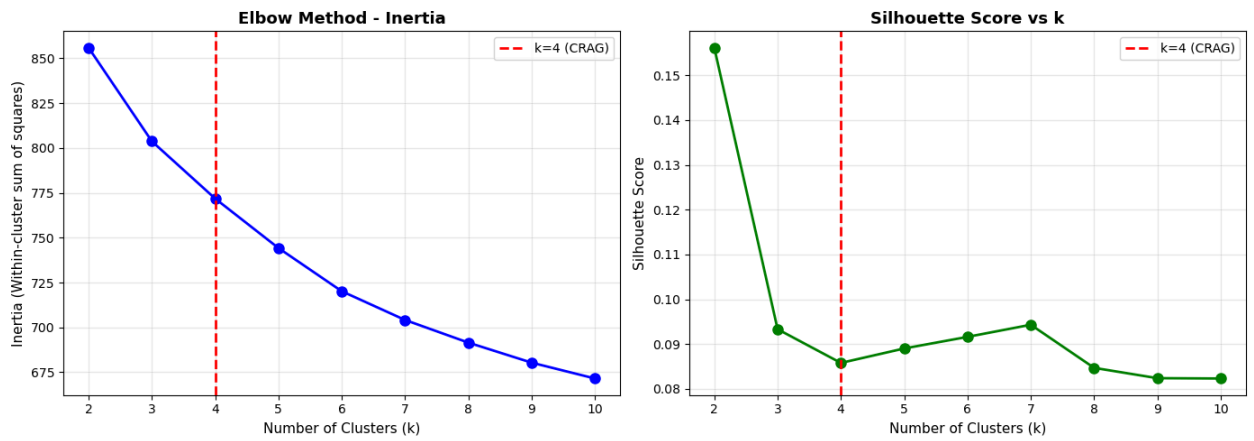
# Silhouette plot
axes[1].plot(k_range, silhouettes, 'go-', linewidth=2, markersize=8)
axes[1].axvline(x=4, color='red', linestyle='--', linewidth=2, label='k=4 (CRA
axes[1].set_xlabel('Number of Clusters (k)', fontsize=11)
axes[1].set_ylabel('Silhouette Score', fontsize=11)
axes[1].set_title('Silhouette Score vs k', fontsize=13, fontweight='bold')
axes[1].grid(alpha=0.3)
axes[1].legend()

plt.tight_layout()
plt.show()

# Find optimal k based on silhouette
optimal_k = k_range[np.argmax(silhouettes)]
print(f"\nBest k based on silhouette score: {optimal_k} (score: {max(silhouett
print(f"CRA paper used k=4 (score: {silhouettes[2]:.4f})")
print(f"\nThe 'elbow' in the inertia plot suggests the point where adding more
print(f"doesn't significantly improve the clustering quality.")

```

Elbow Method analysis for: Apple iPhone 4s 8GB Unlocked Smartphone w/ 8MP Camera, White (Certified Refurbished)
 Testing k from 2 to 10...



Best k based on silhouette score: 2 (score: 0.1561)
CRAG paper used k=4 (score: 0.0858)

The 'elbow' in the inertia plot suggests the point where adding more clusters doesn't significantly improve the clustering quality.